

Advanced JUnit Testing Exercises

Exercise 1: Parameterized Tests

EvenChecker.java :

```
public class EvenChecker {  
  
    public boolean isEven(int number) {  
        return number % 2 == 0;  
    }  
}
```

EvenCheckerTest.java :

```
import org.junit.jupiter.params.ParameterizedTest;  
import org.junit.jupiter.params.provider.ValueSource;  
import static org.junit.jupiter.api.Assertions.*;  
  
public class EvenCheckerTest {  
  
    EvenChecker checker = new EvenChecker();  
  
    @ParameterizedTest  
    @ValueSource(ints = {2, 4, 6, 8, 10})  
    void testEvenNumbers(int number) {  
        assertTrue(checker.isEven(number));  
    }  
}
```

Maven Dependency :

```
<dependency>  
    <groupId>org.junit.jupiter</groupId>  
    <artifactId>junit-jupiter</artifactId>  
    <version>5.10.0</version>  
    <scope>test</scope>  
</dependency>
```

Output :

Tests run: 5

Failures: 0

Exercise 2: Test Suites and Categories

CalculatorTest.java :

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
  
public class CalculatorTest {  
  
    @Test  
    void testAdd() {  
        assertEquals(5, 2 + 3);  
    }  
}
```

```
}  
}
```

EvenCheckerTest.java :

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
  
public class EvenCheckerTest {  
  
    @Test  
    void testEven() {  
        assertTrue(4 % 2 == 0);  
    }  
}
```

AllTests.java :

```
import org.junit.platform.suite.api.SelectClasses;  
import org.junit.platform.suite.api.Suite;  
  
@Suite  
@SelectClasses({  
    CalculatorTest.class,  
    EvenCheckerTest.class  
})  
public class AllTests {  
}
```

Maven Dependencies :

```
<dependency>  
    <groupId>org.junit.jupiter</groupId>  
    <artifactId>junit-jupiter</artifactId>  
    <version>5.10.0</version>  
    <scope>test</scope>  
</dependency>  
  
<dependency>  
    <groupId>org.junit.platform</groupId>  
    <artifactId>junit-platform-suite</artifactId>  
    <version>1.10.0</version>  
    <scope>test</scope>  
</dependency>
```

Output :

```
CalculatorTest ✓  
EvenCheckerTest ✓
```

```
Tests run: 2  
Failures: 0
```

Exercise 3: Test Execution Order

OrderedTests.java :

```

import org.junit.jupiter.api.Order;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestMethodOrder;
import org.junit.jupiter.api.MethodOrderer;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class OrderedTests {

    @Test
    @Order(1)
    void testFirst() {
        System.out.println("First Test");
    }

    @Test
    @Order(2)
    void testSecond() {
        System.out.println("Second Test");
    }

    @Test
    @Order(3)
    void testThird() {
        System.out.println("Third Test");
    }
}

```

Output :

```

First Test
Second Test
Third Test

```

```

Tests run: 3
Failures: 0

```

Exercise 4: Exception Testing

ExceptionThrower.java :

```

public class ExceptionThrower {

    public void throwException() {
        throw new ArithmeticException("Exception Occurred");
    }
}

```

ExceptionThrowerTest.java :

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ExceptionThrowerTest {

    @Test
    void testException() {
        ExceptionThrower et = new ExceptionThrower();
    }
}

```

```

        assertThrows(
            ArithmeticException.class,
            () -> et.throwException()
        );
    }
}

```

Output :

Tests run: 1
Failures: 0

Exercise 5: Timeout and Performance Testing

PerformanceTester.java :

```

public class PerformanceTester {

    public void performTask() {
        for (int i = 0; i < 1000; i++) {
            System.out.print("");
        }
    }
}

```

PerformanceTesterTest.java :

```

import org.junit.jupiter.api.Test;
import java.time.Duration;
import static org.junit.jupiter.api.Assertions.*;

public class PerformanceTesterTest {

    @Test
    void testPerformance() {

        PerformanceTester pt = new PerformanceTester();

        assertTimeout(
            Duration.ofSeconds(1),
            () -> pt.performTask()
        );
    }
}

```

Output :

Tests run: 1
Failures: 0